© 2026 Arkham Technologies Inc.

[Λ] Arkham

AI Engineer Challenge: Research Agent Teams

Objective:
Build a multi-agent research system that can perform research on files or the web, about any given medium-size task.

Details:

Task:
You need to build a multi-agent harness, that should follow the `Planning->PRD->Spec->Implementation` flow, and produce a minimum viable solution that implements the given task (e.g., an API for consulting a mock database, building an MCP server for accessing a remote Obsidian vault)

Technical Requirements:

1. Orchestrator Agent:
    a. The workflow should only be interacted through a streaming response agent.
    b. The orchestrator agent must have access to at least 3 expert agents, the task of each of these are open to your judgment, but will be evaluated.

2. Sub Agents:
    a. Each expert agent needs to have a log trail, exception handling, and a traceback system.
    b. The workflow must have validation rules and an **auditor agent**, it runs a sweep on the steps and validates these rules.

3. Deployment:
    a. The workflow needs an prompt request entry point (can be in terminal or a simple UI), and a response validation, **the user needs to be able to view what was implemented and ask for changes**.

What will be evaluated:

1. You're free to use any existing harness or sdk, but the subagent delegation, task tracking, auditing and re-running logic needs to be implemented and will have the most importance.
2. Accurate logic on each step and decision making will be taken into account.
3. What won't be evaluated is developer's knowledge of specific tools or frameworks, we highly encourage to develop most of the Agent Team orchestration and coordination, and only leave to the tools the LLM servicing and connection. We won't evaluate the

© 2026 Arkham Technologies Inc.

quality of the model responses nor the LLM selection, but we will evaluate how the harness system delegates, communicates and evaluates.

4. <u>You should focus on building a resilient workflow rather than tuning model prompts.</u>

Deliverables:

1. Codebase:
    a. Submit a completed Python codebase that implements the solution, including all necessary functionalities.
    b. Ensure that the code is well-structured, readable, and follows best practices.

Remote Presentation:

a. Prepare an onsite **presentation** to showcase your solution.
b. During the presentation, provide a live demonstration of your system's functionality, including question-answering using the multi-agent system.
c. **Demonstrate the architecture, performance, and potential future improvements of your solution.**

Bonus points:
These are not required, but improve greatly the results **if implemented**:

- An auto-memory fetching and retrieval system.
- Self improvement workflows.
- Agent-hooks during executions.
- Implementation on lower-level harnesses (pi, hermes, openclaw).
- Gateway control.
- Remote access.
- Implementation for open source model.
- Security harnesses (VM virtualization, isolated code execution, black boxed environments).
- Locally executed models.